



**Politecnico  
di Torino**

## POLITECNICO DI TORINO

Mechatronics Engineering

A.a. 2025/2026

Sessione di laurea Mese Anno

## TITOLO DELLA TESI

EVENTUALE SOTTOTITOLO

Relatori:

Name Surname

Name Surname

Candidati:

Emanuele Pruiti



# Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodore sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.



# Indice

|                                                               |     |
|---------------------------------------------------------------|-----|
| <b>Elenco delle figure</b>                                    | VII |
| <b>1 Introduction</b>                                         | 1   |
| 1.1 Goal                                                      | 1   |
| 1.2 Struttura della tesi                                      | 2   |
| <b>2 State of the Art</b>                                     | 3   |
| 2.1 Agricultural row navigation and sensing                   | 3   |
| 2.1.1 Navigation tasks and operating conditions               | 3   |
| 2.1.2 Sensing for local guidance                              | 3   |
| 2.2 Visual perception of crops and inter-row paths            | 4   |
| 2.2.1 Classical image-processing methods                      | 4   |
| 2.2.2 Learning-based segmentation                             | 4   |
| 2.2.3 Training data and model-assisted annotation             | 4   |
| 2.3 From segmentation to a navigation reference               | 5   |
| 2.3.1 Line fitting and crop-edge detection                    | 5   |
| 2.3.2 Column-wise histogram methods                           | 5   |
| 2.3.3 Horizontal strips and adaptive multi-ROI methods        | 5   |
| 2.3.4 Triangle Scan Method                                    | 6   |
| 2.3.5 Comparison of geometric representations                 | 6   |
| 2.4 System integration and experimental evidence              | 6   |
| 2.4.1 Perception and control                                  | 6   |
| 2.4.2 Scope of reported evaluations                           | 8   |
| 2.5 Positioning of this thesis                                | 8   |
| <b>3 Datasets, Segmentation Models and Offline Evaluation</b> | 9   |
| 3.1 Image acquisition and class definition                    | 10  |
| 3.1.1 Image sources and acquisition conditions                | 10  |
| 3.1.2 Semantic classes and labeling targets                   | 10  |
| 3.2 Lettuce, tomato, and cucumber datasets                    | 10  |
| 3.2.1 Lettuce dataset                                         | 10  |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 3.2.2    | SAM-assisted multi-crop datasets and label configurations . . .   | 10        |
| 3.2.3    | Tomato dataset . . . . .                                          | 11        |
| 3.2.4    | Cucumber dataset . . . . .                                        | 11        |
| 3.3      | Manual and SAM 3-assisted labeling . . . . .                      | 11        |
| 3.3.1    | Manual labeling procedure . . . . .                               | 11        |
| 3.3.2    | Model-assisted labeling procedure . . . . .                       | 11        |
| 3.4      | Labeling quality and dataset partitioning . . . . .               | 12        |
| 3.4.1    | Label inspection and revision criteria . . . . .                  | 12        |
| 3.4.2    | Training, validation, and test sets . . . . .                     | 13        |
| 3.5      | FCN, DeepLabV3, and LR-ASPP: initialization and fine-tuning . .   | 14        |
| 3.5.1    | Selection of the TorchVision implementations . . . . .            | 14        |
| 3.5.2    | Pretrained weights and adaptation of the output classes . . .     | 14        |
| 3.5.3    | Backbone freezing and fine-tuning . . . . .                       | 15        |
| 3.6      | Training configuration and checkpoint selection . . . . .         | 16        |
| 3.6.1    | Preprocessing and data augmentation . . . . .                     | 16        |
| 3.6.2    | Loss function and optimization settings . . . . .                 | 16        |
| 3.6.3    | Validation and checkpoint selection . . . . .                     | 16        |
| 3.7      | Offline architecture comparison on manually annotated lettuce . . | 16        |
| 3.7.1    | Evaluation protocol and inference-time measurement . . . .        | 16        |
| 3.7.2    | Accuracy and inference-time results . . . . .                     | 17        |
| 3.7.3    | Motivation for adopting LR-ASPP . . . . .                         | 19        |
| 3.8      | Offline comparison of supervision strategies . . . . .            | 19        |
| 3.8.1    | Training configurations and evaluation targets . . . . .          | 19        |
| 3.8.2    | Metrics and interpretation of the comparison . . . . .            | 20        |
| 3.8.3    | Results on the full lettuce evaluation set . . . . .              | 21        |
| 3.8.4    | Results on the selected operational subset . . . . .              | 21        |
| 3.8.5    | Discussion and limitations of the offline comparison . . . .      | 21        |
| 3.9      | Segmentation configuration selected for robotic integration . . . | 22        |
| 3.10     | ONNX export and inference configuration . . . . .                 | 22        |
| 3.10.1   | Model export . . . . .                                            | 22        |
| 3.10.2   | Inference inputs and outputs . . . . .                            | 22        |
| <b>4</b> | <b>Geometric Extraction of the Local Navigation Reference</b>     | <b>23</b> |
| 4.1      | Input representation and edge-line models . . . . .               | 23        |
| 4.2      | Region of interest and mask preparation . . . . .                 | 24        |
| 4.3      | Horizontal strips and candidate intervals . . . . .               | 25        |
| 4.3.1    | Strip construction and column projection . . . . .                | 25        |
| 4.3.2    | Method-specific local support . . . . .                           | 26        |
| 4.4      | Initialization and bottom-up association . . . . .                | 26        |
| 4.4.1    | Method 3: lowest supported seed . . . . .                         | 26        |
| 4.4.2    | Method 4: edge-support seed and gap-limited association . .       | 27        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 4.5      | Separate robust fitting of the corridor edges . . . . .           | 28        |
| 4.5.1    | Excluding image-boundary observations . . . . .                   | 28        |
| 4.5.2    | Residual-based reweighting . . . . .                              | 28        |
| 4.6      | Quality indicators, validation, and image-space output . . . . .  | 29        |
| 4.6.1    | Method 3: centerline support after rasterization . . . . .        | 29        |
| 4.6.2    | Method 4: geometric checks and a composite score . . . . .        | 29        |
| 4.6.3    | Interpretation of status and confidence . . . . .                 | 30        |
| 4.7      | Default configurations and comparison . . . . .                   | 31        |
| 4.8      | Effect of strip height and number . . . . .                       | 32        |
| <b>5</b> | <b>Conclusions and Future Work</b>                                | <b>35</b> |
| 5.1      | Thesis objectives and main contributions . . . . .                | 35        |
| 5.2      | Conclusions on dataset preparation and automatic annotation . . . | 36        |
| 5.3      | Selection of the segmentation model for the robot . . . . .       | 36        |
| 5.3.1    | Architecture selection: accuracy and inference time . . . . .     | 36        |
| 5.3.2    | Supervision and target-class selection . . . . .                  | 36        |
| 5.4      | Conclusions on geometric reference extraction . . . . .           | 37        |
| 5.5      | Conclusions from robot field validation . . . . .                 | 37        |
| 5.5.1    | Test conditions and scope of the evidence . . . . .               | 37        |
| 5.5.2    | Row following and task completion . . . . .                       | 37        |
| 5.5.3    | Reference loss, rejection, and robot response . . . . .           | 38        |
| 5.5.4    | Processing latency and closed-loop operation . . . . .            | 38        |
| 5.6      | Limitations and validity of the findings . . . . .                | 38        |
| 5.7      | Future work . . . . .                                             | 39        |
| 5.7.1    | Dataset expansion and annotation verification . . . . .           | 39        |
| 5.7.2    | Geometric extraction and temporal integration . . . . .           | 39        |
| 5.7.3    | Further robot trials and operational coverage . . . . .           | 39        |
| 5.8      | Final remarks . . . . .                                           | 39        |
|          | <b>Bibliografia</b>                                               | <b>40</b> |

# Elenco delle figure

# Capitolo 1

## Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Porttitor eget dolor morbi non arcu risus quis varius. Libero id faucibus nisl tincidunt. Neque laoreet suspendisse interdum consectetur libero id faucibus nisl tincidunt. Scelerisque in dictum non consectetur a erat. Leo a diam sollicitudin tempor id eu. Sodales ut eu sem integer vitae justo eget magna fermentum. A cras semper auctor neque vitae. Cursus euismod quis viverra nibh cras pulvinar. Mi tempus imperdiet nulla malesuada pellentesque elit eget gravida cum. Dictum at tempor commodo ullamcorper a lacus vestibulum sed. Ultricies mi eget mauris pharetra. In pellentesque massa placerat dui ultricies lacus. Fringilla phasellus faucibus scelerisque eleifend donec pretium vulputate.

Aliquam sem fringilla ut morbi tincidunt augue interdum. Risus at ultrices mi tempus imperdiet nulla malesuada pellentesque. Semper quis lectus nulla at volutpat. Nullam non nisi est sit amet facilisis. Eget velit aliquet sagittis id consectetur purus ut faucibus pulvinar. Ultricies tristique nulla aliquet enim tortor at auctor urna nunc. Pharetra diam sit amet nisl suscipit adipiscing bibendum est ultricies. Amet facilisis magna etiam tempor. Nunc non blandit massa enim nec dui nunc mattis. At ultrices mi tempus imperdiet nulla malesuada pellentesque. Cursus metus aliquam eleifend mi in nulla posuere. In eu mi bibendum neque egestas congue quisque. Augue eget arcu dictum varius dui at consectetur.

### 1.1 Goal

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla

phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodos egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

## **1.2    Struttura della tesi**

Lorem ipsum dolor sit amet:

- onsectetur adipiscing elit,
- sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
- Tristique senectus et netus et malesuada fames ac turpis.

## Capitolo 2

# State of the Art

This chapter reviews vision-based methods for agricultural row navigation, focusing on the conversion of image observations into a local guidance reference. It covers visual perception, geometric extraction, and the integration of these stages with robot control. Dataset preparation and the implementation used in this thesis are described in Chapters 3 and 4.

## 2.1 Agricultural row navigation and sensing

### 2.1.1 Navigation tasks and operating conditions

Vision-based agricultural navigation uses image features to identify crop rows, crop boundaries, or the soil region between rows, and then converts them into a reference for the robot [1]. These are related but distinct tasks: following the center of a crop row, staying in the inter-row path, and detecting the boundary between standing crop and harvested ground do not produce the same visual reference. The reviewed studies address greenhouse cucumber rows, sugar-beet fields, vineyards, orchards, and wheat or rapeseed fields [2, 3, 4, 5, 6].

Field appearance varies with illumination, shadows, weeds, plant size, occlusion, and gaps in the rows. These conditions affect both segmentation and the geometric reference derived from it [3, 5].

### 2.1.2 Sensing for local guidance

GNSS can provide global position outdoors, but crop canopies may obstruct satellite signals. LiDAR, stereo, and RGB-D cameras provide depth or three-dimensional structure; their suitability depends on the field environment, sensor configuration, and cost. A monocular camera is less complex and provides appearance information, but does not directly measure depth [1, Sections 2.1–2.3]. Navone et al. use RGB-D

sensing for local guidance in dense rows where vegetation can also obstruct GNSS reception [5].

This thesis focuses on camera-based local guidance. It does not address global localization or reconstruct the field in three dimensions.

## **2.2 Visual perception of crops and inter-row paths**

### **2.2.1 Classical image-processing methods**

Classical pipelines combine image filtering, thresholding, and application-specific geometric rules. Mendez et al. convert vineyard images to grayscale, apply Gaussian filtering and Otsu thresholding, select the largest blob, and use column pixel counts to estimate the row center [4, Sections 3.4–4]. Such methods can be effective in the conditions for which their parameters were selected, but changes in illumination, weeds, and crop appearance can require retuning [6, Section 2].

### **2.2.2 Learning-based segmentation**

Semantic segmentation assigns a class to each pixel, but the label definition determines what the geometric stage can recover. De Silva et al. train U-Net to predict narrow crop-row masks in sugar-beet images; the labels represent row structure rather than individual plant pixels or traversable soil [3, Sections 3.3 and 4.1]. Navone et al. instead predict a binary vegetation mask from RGB images using a MobileNetV3 backbone and an LR-ASPP head, then combine that output with depth for reference extraction [5, Sections 3.3 and 4.1].

The distinction matters for this work: its target is the soil region between crop rows, not a crop-row line or a vegetation mask. A segmentation model therefore supplies a candidate region; the geometric stage must still select the corridor and construct the guidance reference.

### **2.2.3 Training data and model-assisted annotation**

Pixel-level annotation is time-consuming, and agricultural datasets must represent changes in crop, growth stage, and lighting. De Silva et al. include 11 field variations in their dataset [3, Section 3]. Navone et al. initialize their segmentation model with ImageNet weights and train using synthetic AgriSeg images supplemented by 100 real crop images [5, Section 4.1, Table 1]. Pretraining and the choice of training images are separate design choices.

Segment Anything Model 3 (SAM 3) supports text- and image-prompted concept segmentation. Its training data were assembled using a data engine with human

and automated quality checks [7, Sections 2 and 4]. In this thesis, SAM 3 is used to prepare labels offline, not as the network running during robot navigation. The quality of these masks and the time required to review or correct them are evaluated separately in Chapter 3.

## 2.3 From segmentation to a navigation reference

The geometric stage reduces a mask to a quantity the controller can use. Depending on the representation and method, the output may be a horizontal image position, a straight line, a crop-edge estimate, or a curve. These outputs describe different tasks and coordinate systems.

### 2.3.1 Line fitting and crop-edge detection

Chen et al. estimate a path line for greenhouse cucumber rows from navigation points selected in plant–soil segmentation. Their prediction-point Hough transform uses a least-squares estimate to restrict the line search and is compared with traditional Hough and least-squares fitting [2, Sections 2–3].

Kneip et al. detect crop–ground transitions in scanlines of a stereo elevation map, then fit a linear crop-edge model using a Huber M-estimator and iteratively reweighted least squares. Their output is a crop boundary in ground-plane coordinates, not an image-space estimate of the inter-row path [6, Sections 6–7].

### 2.3.2 Column-wise histogram methods

Histogram methods aggregate mask pixels by image column. Mendez et al. estimate the row center from the five columns with the largest pixel counts in a thresholded background region [4, Section 3.4.3]. Navone et al. use a different input: a temporally combined vegetation mask refined with depth. SegMin smooths its column histogram and selects the minimum; SegMinD weights the vegetation mask by normalized inverse depth before calculating the histogram [5, Section 3]. In both cases the output is a horizontal position, not a full path line.

### 2.3.3 Horizontal strips and adaptive multi-ROI methods

Li et al. combine a Faster-U-net inter-row-path mask with an adaptive multi-ROI procedure. Horizontal regions supply ordered local points, which are used as control points for a multisegment cubic B-spline. Their experiments use 40-pixel search regions and discuss the trade-off between the number of points and fitting behavior [8, Section 2.2.3, Figure 9].

This is the closest geometric comparison for the present work because both pipelines select local evidence from horizontal image regions. The representation and fitted model differ: Li et al. use an inter-row-path mask and a spline, whereas this thesis estimates a local straight reference from the soil corridor between parallel crop rows.

### 2.3.4 Triangle Scan Method

De Silva et al. propose the Triangle Scan Method (TSM) for crop-row masks. It first scans an upper horizontal band to find an anchor column, using a preset anchor if the scan fails a threshold. Candidate lines from that anchor to positions along the base of a triangular region are then evaluated; the line with the greatest mask support defines the central crop row. Its two endpoints provide position and angular errors for visual servoing [3, Section 4.2, Algorithm 1].

TSM is not the same as extracting one point from each of several horizontal strips: the upper scan establishes an anchor for a bounded line search. It also operates on crop-row masks, so its output is not directly equivalent to a reference extracted from a soil mask.

### 2.3.5 Comparison of geometric representations

Table 2.1 summarizes the input representation and geometric output of the methods discussed above.

The methods are not directly ranked by this comparison: their inputs and outputs differ. Li et al. provide the closest structural precedent for strip-based point selection, while TSM illustrates a line search for a crop-row representation. Neither method uses the same soil-corridor mask and straight-line model as the pipeline developed in this thesis.

## 2.4 System integration and experimental evidence

### 2.4.1 Perception and control

Navigation systems use the extracted reference to generate motion commands. Navone et al. convert the displacement of their histogram reference into linear and angular velocity commands and smooth them with an Exponential Moving Average [5, Section 3.4]. Mendez et al. use proportional control based on row-center displacement and the distribution of background pixels [4, Section 3.5]. De Silva et al. combine the central row's angular and positional errors in a proportional visual-servoing controller for simulation [3, Section 4.3].

**Tabella 2.1:** Perceptual inputs and geometric references in selected agricultural navigation studies.

| Study               | Perceptual input                              | Geometric operation                                           | Output                         |
|---------------------|-----------------------------------------------|---------------------------------------------------------------|--------------------------------|
| Chen et al. [2]     | Plant–soil segmentation and navigation points | Restricted Hough search around a predicted point              | Image-space path line          |
| Kneip et al. [6]    | Stereo elevation map                          | Scanline transitions and robust linear fit                    | Ground-plane crop edge         |
| Mendez et al. [4]   | Thresholded background region                 | Column counts and mean of the five largest                    | Horizontal row-center position |
| Navone et al. [5]   | Vegetation mask with depth                    | Smoothed column histogram; inverse-depth weighting in SegMinD | Histogram-minimum position     |
| Li et al. [8]       | Predicted inter-row-path mask                 | Adaptive horizontal regions and cubic B-spline                | Curved image-space path        |
| De Silva et al. [3] | Predicted crop-row mask                       | Upper anchor and triangular line search                       | Central crop-row line          |

### 2.4.2 Scope of reported evaluations

The studies evaluate different stages of navigation. Kneip et al. assess crop-edge detection on recorded harvesting data [6, Section 8]; Chen et al. report angular errors and processing times for image-based path fitting [2, Section 3.6, Tables 6–7]. De Silva et al. evaluate row detection on real images and visual servoing in simulation, with preliminary real-world row following also reported [3, Sections 5.2–5.3 and 6, Table 5]. Mendez et al. report closed-loop vineyard trials, measuring centering error from the distances to the two sides of the row [4, Sections 3.6 and 4]. Navone et al. test in simulation and crop fields, using LiDAR and attitude measurements to assess displacement and heading [5, Sections 4.3–4.4].

Segmentation IoU, image-space line error, field tracking error, and processing time measure different parts of the system. Results from these studies therefore provide context, not a common performance benchmark.

## 2.5 Positioning of this thesis

This thesis studies a camera-based pipeline that segments plants and the soil corridor, selects the relevant inter-row region, and estimates a local guidance line for navigation between parallel crop rows. Its geometric stage uses horizontal strips to select candidate intervals and fits the two corridor boundaries separately. This shares the local-selection structure of Li et al.’s multi-ROI method, but uses a straight image-space reference rather than a spline. The experiments in the following chapters evaluate the segmentation models, geometric extractors, and integrated robot at their respective stages.

## Capitolo 3

# Datasets, Segmentation Models and Offline Evaluation

The segmentation study began with a comparison of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images. Manual segmentation was time-consuming, so this first comparison was limited to lettuce rather than repeated for every crop. Its outcome led to the adoption of LR-ASPP.

SAM 3 was then used to generate training labels for lettuce, tomato, and cucumber images. The resulting multi-crop models differed in their output classes: plants and path plus background, four classes including covering material, or path and background alone. These models were evaluated against the manual lettuce masks. The crop coverage of the training data therefore differs from that of the reported evaluation set.

This chapter describes the datasets and training procedure, followed by the offline comparisons that motivate the configuration selected for the robot. The performance of the trained networks is considered separately from the quality of the SAM 3 annotations themselves. The architecture results are reported from the available validation exports; the comparisons of supervision strategies still require checks on evaluation provenance. Geometric and closed-loop field experiments are discussed later in the thesis.

## 3.1 Image acquisition and class definition

### 3.1.1 Image sources and acquisition conditions

### 3.1.2 Semantic classes and labeling targets

The four-class configurations—the lettuce models used to select the segmentation architecture and the four-class SAM 3 configuration—use background, plants, soil, and covering material.

The two-foreground-class SAM 3 configuration labels plants and path; with background, it has three output labels. The path-only SAM 3 configuration is binary, with background and path as its two output labels. For robotic integration, the relevant output is the path mask. Segmenting soil alone does not identify a unique navigation corridor or establish that it is obstacle-free. The geometric methods in Chapter 4 use the mask to construct a local image-space reference.

## 3.2 Lettuce, tomato, and cucumber datasets

### 3.2.1 Lettuce dataset

The manually annotated lettuce dataset, identified as **Data**, contains 762 image-mask pairs distributed across eight folders. Its masks were prepared from the manual annotations stored in `annotations.json`. The same manual reference is used for the initial architecture comparison and for evaluating the models trained with SAM 3-generated multi-crop labels. Automatically annotated lettuce images belong to the latter training datasets, not to separate lettuce-only training configurations.

### 3.2.2 SAM-assisted multi-crop datasets and label configurations

The automatically supervised models use images spanning all the crops considered in this work. The change between configurations concerns the semantic targets to be segmented, not a restriction to a different crop: plants and path plus background, plants and path and covering material plus background, or path plus background alone. The exact image lists, partitions, and dataset export associated with each checkpoint must nevertheless be retained; common crop coverage does not by itself verify identical sample membership across training runs.

The binary `allPlantsSAM` model was trained on `Data_allPlantsSAM2`. Its automatic masks describe only path, with all remaining pixels assigned to background. This formulation focuses the output on the target region used by the geometric extractor, rather than on classification of every scene component. The

dataset-directory identifiers for the other label configurations must be established from their training records, not inferred from the names of the lettuce evaluation files.

The lettuce evaluation examines these multi-crop models using manually annotated lettuce images. Multi-crop training does not establish that those evaluation images are unseen. Possible image overlap, duplicates, or related acquisitions between training and evaluation have not yet been checked; conclusions about generalization remain subject to that verification.

### **3.2.3 Tomato dataset**

The tomato images were labeled using SAM 3 rather than the manual procedure used for the initial lettuce comparison. They contribute to the automatically annotated multi-crop training data. LR-ASPP was adopted after the comparison of the three candidate architectures on lettuce; no separate FCN, DeepLabV3, and LR-ASPP comparison was performed on tomato.

### **3.2.4 Cucumber dataset**

SAM 3 was also used to label the cucumber images, which contribute to the automatically annotated multi-crop training data. As for tomatoes, the architecture choice followed the comparison on manually annotated lettuce, rather than a separate comparison on cucumber.

## **3.3 Manual and SAM 3-assisted labeling**

### **3.3.1 Manual labeling procedure**

The lettuce images were labeled manually before training the three candidate architectures. The masks derived from `annotations.json` are adopted as the ground truth for evaluating the trained segmentation models on lettuce. They can also serve as the reference for a separate direct comparison with SAM 3-generated masks. Using these annotations as ground truth defines the evaluation reference; it does not imply that manual annotation is intrinsically error-free.

### **3.3.2 Model-assisted labeling procedure**

The time required for manual labeling motivated the use of SAM 3 to prepare the multi-crop datasets. Initial use suggested faster mask generation, but the total annotation effort has not been measured in a systematic comparison that includes inspection and correction.

The SAM 3-assisted labeling procedure covers lettuce, tomato, and cucumber images for the multi-crop training configurations. In this workflow, automatic segmentation is used to prepare labels offline; it is distinct from the segmentation model used during navigation.

For the lettuce images annotated with two foreground classes, the reported text prompts are `plants` and `central farm path`. The four-class labeling additionally uses `plastic mulch film` for covering material. Background accounts for the remaining label in each configuration. These reported lettuce prompts do not establish the exact prompts used for every crop in the multi-crop datasets. The binary configuration retains only the path annotation; crop-specific prompts and mask-composition settings for each configuration must be documented from the corresponding labeling runs.

These text descriptions define annotation targets, but do not specify how overlapping masks are resolved. The available `sam3_autolabel_3class.py` is configured for tomatoes with the prompts `tomato plant` and `soil`; it gives plant pixels precedence over soil pixels when composing the mask. Its settings should not be attributed to the lettuce labeling runs without checking their implementation. In particular, an annotation of visible soil and a prompt requesting the central farm path can target different regions. Agreement with the manual soil class must therefore be assessed rather than assumed.

Initial visual inspection suggested good results on images considered relatively simple, although this impression still requires systematic evaluation. Inspection of the lettuce masks also revealed cases in which the automatic procedure identified only some of the plants visible in the image. Other masks were less precise than their manual counterparts. Errors were observed particularly in images containing small lettuce plants, while distant plants were a recurring difficulty among the images with labeling errors. These are qualitative observations from the inspected images, not yet measurements of error frequency or evidence that the same behavior occurs to the same extent in the other crops.

## 3.4 Labeling quality and dataset partitioning

### 3.4.1 Label inspection and revision criteria

The planned quantitative assessment evaluates the quality of the SAM 3-generated lettuce masks against the manual ground-truth masks. The intended measure is the intersection over union for the lettuce class, comparing the automatically labeled pixels with the corresponding manual reference. This comparison is intended to quantify the quality of SAM 3 labeling on the evaluated lettuce images, not its performance across all crops or applications. The exact evaluation set and

aggregation procedure have not yet been fixed, and no numerical labeling-quality results are reported here.

This assessment must be kept separate from the comparison of the trained segmentation models. Agreement between two sets of labels does not directly measure the performance of a network trained on either set. The additional models trained on automatic multi-crop labels address a separate question: how the resulting segmentation networks perform against a common manual reference. Their performance also depends on the training images, target classes, model configuration, and optimization. The difference between two networks cannot therefore be attributed exclusively to annotation quality when other factors differ.

### **3.4.2 Training, validation, and test sets**

The reported training runs use an 80% training, 10% validation, and 10% test partition with seed 42. The available `train_model.py` implements these proportions by assigning the integer parts of the training and validation counts and leaving the remaining samples for test. Applied to 762 pairs, this rule gives 609 training, 76 validation, and 77 test samples. These counts describe the intended manual-dataset partition; the saved indices and image lists must establish the actual membership of each evaluated split.

The test set is not used for gradient-based parameter updates. Checkpoint selection uses validation rather than test performance. A common seed alone does not ensure that differently sized or ordered datasets yield identical image partitions. Corresponding image identities must therefore be checked when comparing the manually supervised lettuce model with the automatically supervised multi-crop models, including whether a lettuce evaluation image appears in an automatic model’s training set.

The reported operational evaluation selects 42 images from the manual test split belonging to `insalata_grande1`, `insalata_grande2`, and `insalata_medial1`. These folders represent the row conditions selected for the robot application. The 42 images are a subset of the reported 77 test images, not an additional independent test set. Performance on this subset describes those selected conditions, rather than the full range of lettuce images.

## 3.5 FCN, DeepLabV3, and LR-ASPP: initialization and fine-tuning

### 3.5.1 Selection of the TorchVision implementations

FCN, DeepLabV3, and LR-ASPP were chosen from the implementations available in TorchVision. All three were trained on the manually annotated lettuce dataset. Their selection was practical, rather than the outcome of a comparison with every available architecture. Accuracy and inference time were then used to choose among these candidates, as discussed in Section 3.7.

### 3.5.2 Pretrained weights and adaptation of the output classes

The initialization and training strategy described below follows the available implementation in `models.py` and `train_model.py`. The current training script is configured for the tomato dataset; its correspondence with the earlier lettuce runs remains to be confirmed before these settings are associated with their experimental results.

The implementation separates each network into a backbone, which extracts image features, and a segmentation head, which produces class scores at each output location. Transfer learning is implemented by loading pretrained parameters and adapting the output layers to the target labeling task, rather than initializing every network parameter from scratch. The model construction function accepts the number of output classes as an argument; the available training script explicitly sets this number to three.

For FCN and DeepLabV3, the selected implementations use a ResNet-50 backbone. They are constructed with the default pretrained segmentation weights specified by `FCN_ResNet50_Weights` and `DeepLabV3_ResNet50_Weights`, respectively. After these weights are loaded, the final convolution of the main classifier is replaced by a new  $1 \times 1$  convolution with one output channel per target class. The remaining pretrained layers are retained. Both models also include an auxiliary classifier, whose final convolution is replaced in the same way. Thus, the output layers associated with the original classes are newly initialized, while the preceding layers retain their pretrained parameters.

LR-ASPP is initialized differently. The implementation uses a MobileNetV3-Large backbone and requests its default pretrained weights through the `weights_backbone` argument. However, the `weights` argument of the complete segmentation model is set to `None`. Consequently, the backbone is pretrained, whereas the LR-ASPP segmentation head is newly initialized for the requested number of classes.

The code does not load a pretrained LR-ASPP segmentation checkpoint and then replace only its final layers.

This distinction is relevant to the interpretation of the model comparison. The three configurations differ not only in architecture, but also in the extent of their pretrained initialization: FCN and DeepLabV3 retain pretrained segmentation-head layers, whereas LR-ASPP starts with a new segmentation head. Their results therefore describe the implemented configurations, not an isolated comparison of architectures under identical initialization conditions.

### 3.5.3 Backbone freezing and fine-tuning

Training is organized into two stages. Before the first epoch, the backbone parameters are excluded from gradient-based updates by setting their `requires_grad` attribute to `False`. The complete main classifier remains trainable, as does the auxiliary classifier when present. This first stage therefore updates the segmentation heads, not just the newly replaced output convolutions of FCN and DeepLabV3.

For the first ten epochs, AdamW is constructed using only the trainable parameters, with an initial learning rate of  $10^{-3}$  and weight decay of  $10^{-4}$ . A cosine-annealing scheduler is stepped after each epoch, with a minimum learning rate of  $10^{-6}$ . Its initial horizon is set to the configured total of 200 epochs, although this scheduler is replaced at the transition to the second stage.

At the end of epoch ten, the script enables gradients for all backbone parameters. Fine-tuning of the backbone therefore begins at epoch eleven, together with continued training of the segmentation heads. A new AdamW optimizer is constructed with separate parameter groups: the backbone starts this stage at a learning rate of  $10^{-5}$ , while the main and, where present, auxiliary classifiers start at  $10^{-4}$ . Weight decay remains  $10^{-4}$ . The backbone learning rate is thus lower than that of the heads, allowing the pretrained features to be adapted with smaller learning-rate settings during this stage.

The optimizer is recreated rather than extended in place, so its accumulated state from the first stage is not transferred to the second. The model parameters themselves are retained at the transition. A new cosine-annealing scheduler is also created for the remaining 190 epochs, again with a minimum learning rate of  $10^{-6}$ . The script configures 200 epochs in total; the early-stopping block is commented out and does not terminate training in the active implementation.

Here, backbone freezing refers specifically to parameter updates. The training loop calls `model.train()` without separately placing the backbone in evaluation mode. Its batch-normalization running statistics therefore continue to update during training, including the first stage. The backbone is not completely fixed in the sense of preserving all of its internal state. Validation is instead performed with `model.eval()` and gradient computation disabled.

This two-stage procedure first adapts the segmentation heads and then jointly fine-tunes the backbone and heads. The loss function, data transformations, and checkpoint-selection criterion are addressed in Section 3.6, separately from the initialization and parameter-update schedule described here.

## 3.6 Training configuration and checkpoint selection

### 3.6.1 Preprocessing and data augmentation

### 3.6.2 Loss function and optimization settings

### 3.6.3 Validation and checkpoint selection

In the available training implementation, validation is performed without gradient updates after each epoch. A checkpoint is saved as `best.pth` whenever the validation loss is lower than all previous values; `last.pth` stores the latest training state. Selection therefore uses minimum validation loss, not maximum test IoU or simply the last epoch. The script records the epoch, training and validation losses, split proportions, and seed with the checkpoint, and saves split indices separately.

This describes the supplied training script, currently configured for tomatoes. The corresponding records must confirm the criterion and settings used for the manual lettuce checkpoint and each SAM-supervised multi-crop checkpoint evaluated on lettuce. The differing checkpoint epochs in the comparison files do not, on their own, establish either a common training duration or a causal explanation for differences in segmentation accuracy.

## 3.7 Offline architecture comparison on manually annotated lettuce

This comparison addresses the architecture choice before changing the annotation procedure or crop coverage. FCN, DeepLabV3, and LR-ASPP were trained and evaluated using the manually annotated lettuce dataset. The implemented initializations differ as described in Section 3.5.2; the comparison concerns these trained configurations rather than architecture alone under identical pretraining conditions.

### 3.7.1 Evaluation protocol and inference-time measurement

The comparison uses the three CSV records stored in `training insalata`: the FCN, DeepLabV3, and LR-ASPP `*_validation_metrics.csv` exports. Each

contains aggregate metrics, the checkpoint epoch, and IoU values for background, plants, soil, and covering material. The reported mIoU agrees with the arithmetic mean of these four class IoUs, including background. The records are presented here as validation exports, not as results of an independent final test.

The CSV files do not identify the evaluated images or timing hardware. The supplied `validation_model.py` selects validation indices, aggregates a confusion matrix, and measures the model forward pass and `argmax` at batch size one, with CUDA synchronization before and after the timed block when a GPU is available. It contains no explicit warm-up stage. However, that script is currently configured for tomato images and three classes, whereas these exports contain four classes. The original lettuce evaluation configuration must therefore confirm the image lists, hardware, resolution, and timing procedure before the recorded speed ratios are treated as a fully controlled benchmark.

### 3.7.2 Accuracy and inference-time results

Table 3.1 reports the exported metrics. Inference times have been converted from seconds to milliseconds. The FPS values equal the reciprocal of the mean recorded time; they are not measurements of the complete robot processing or control rate. Checkpoint epochs identify the evaluated weights, not training duration.

**Tabella 3.1:** Architecture comparison from the manual lettuce validation exports. Timing values are those recorded in the CSV files, not end-to-end robot latency.

| Model     | Epoch | mIoU   | Path IoU | Mean time<br>(ms) | FPS   |
|-----------|-------|--------|----------|-------------------|-------|
| FCN       | 38    | 0.9232 | 0.9621   | 58.24             | 17.17 |
| DeepLabV3 | 43    | 0.9228 | 0.9651   | 86.87             | 11.51 |
| LR-ASPP   | 194   | 0.9108 | 0.9546   | 10.41             | 96.07 |

FCN and DeepLabV3 achieve mIoU values of 0.9232 and 0.9228, respectively, compared with 0.9108 for LR-ASPP. Using the unrounded CSV values, the LR-ASPP deficit is 1.24 percentage points relative to FCN and 1.19 relative to DeepLabV3. Its mean recorded time is 10.41 ms, compared with 58.24 ms and 86.87 ms. These timings correspond to a 5.60-fold reduction relative to FCN and an 8.35-fold reduction relative to DeepLabV3.

Table 3.2 provides the class-level comparison. LR-ASPP has lower IoU than both alternatives in each class, although its soil/path IoU remains 0.9546. The data support a trade-off between accuracy and latency, not a claim of equal segmentation accuracy.

**Tabella 3.2:** Per-class IoU from the same manual lettuce validation exports. Plants, soil, and covering material correspond to `pianta`, `suolo`, and `copertura` in these files.

| Model     | Background | Plants | Soil/path | Covering material |
|-----------|------------|--------|-----------|-------------------|
| FCN       | 0.9464     | 0.9280 | 0.9621    | 0.8565            |
| DeepLabV3 | 0.9464     | 0.9277 | 0.9651    | 0.8519            |
| LR-ASPP   | 0.9394     | 0.9120 | 0.9546    | 0.8372            |

### 3.7.3 Motivation for adopting LR-ASPP

LR-ASPP was adopted because the substantial reduction in recorded inference time justified accepting an mIoU approximately 1.2 percentage points below the other two configurations. It was not the most accurate model, but offered the practical accuracy–latency compromise sought for robot perception. FCN and DeepLabV3 had higher IoU but substantially longer recorded inference times. Subsequent SAM-supervised multi-crop training therefore used LR-ASPP while varying the output label set.

This choice is based on the available offline comparison. The records do not establish a hardware-independent speed ranking or demonstrate that the integrated robot meets its control timing requirements. Those requirements must be assessed using the deployed model and the complete perception and control pipeline.

## 3.8 Offline comparison of supervision strategies

### 3.8.1 Training configurations and evaluation targets

Table 3.3 summarizes the four training configurations considered in this comparison. This study is separate from the initial comparison of the three candidate architectures. The manual configuration is trained on lettuce only, whereas all three SAM-supervised configurations use multi-crop data and differ in their output label sets. The model names identify supervision and targets; they do not mean that SAM 3 itself is executed during robot navigation. Each dataset export, image partition, architecture, and checkpoint must be retained in the experiment records.

**Tabella 3.3:** Training scope and target labels for the comparison of manual and SAM 3-generated supervision. Class counts include background; evaluation uses the manual lettuce reference.

| Configuration                                 | Training scope               | Output labels                               |
|-----------------------------------------------|------------------------------|---------------------------------------------|
| Manual                                        | Lettuce only ( <b>Data</b> ) | Background, plants, path, covering material |
| SAM-generated, two foreground classes         | Multi-crop                   | Background, plants, path                    |
| SAM-generated, four classes                   | Multi-crop                   | Background, plants, path, covering material |
| SAM-generated, binary ( <b>allPlantsSAM</b> ) | Multi-crop                   | Background, path                            |

Five evaluations on manually annotated lettuce images are organized around these configurations:

1. the manual and automatically supervised four-class models, evaluated on all four categories over the full evaluation split;
2. the manual model and the automatically supervised model with two foreground classes, evaluated on plants and path;
3. the manual and SAM-supervised binary models, evaluated only on path;
4. the same path-only comparison on the selected operational subset;
5. the four-class comparison on that operational subset.

The second evaluation uses a separate automatically supervised checkpoint, not merely a different average of the four-class model’s metrics. The subset evaluations reuse their respective trained models; changing the evaluation images does not constitute retraining.

Manual masks supply the ground truth for all comparisons. For path-only scoring, the target is the manual soil class and the predicted positive region is the corresponding path output of each model. This definition allows a four-class and a binary model to be evaluated on the same positive class. The treatment of other manual classes, including covering material, must be explicit in the evaluation implementation; they must not be silently omitted when defining the pixels used for scoring.

### 3.8.2 Metrics and interpretation of the comparison

For a class  $c$ , intersection over union is defined as

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}, \quad (3.1)$$

where true positives, false positives, and false negatives are defined against the manual reference. The available `metrics.py` computes class IoUs from a confusion matrix. The aggregation used by each exported comparison must nevertheless be confirmed: computing IoU from pooled pixel counts and averaging image-wise IoUs are not interchangeable.

The four-class mIoU averages background, plants, path, and covering material. The two-foreground-class comparison instead averages only plants and path; the path-only comparison reports a single class IoU. These averages concern different targets and are not used as directly comparable overall rankings. Path IoU is the common task-specific measure, provided the evaluation images and treatment of ground-truth pixels are aligned.

Before completing the numerical tables in this section as full-test results, the evaluation provenance must be resolved: the currently available four-class and

two-foreground-class CSV files have `val` in their names and contain values different from the reported full-test summary. Their filenames alone do not prove which split was used. Saved image lists, scripts, and logs are needed to reconcile those exports with the reported 77-image test protocol. Inference-time comparisons likewise require consistent hardware, preprocessing, and timing boundaries.

### 3.8.3 Results on the full lettuce evaluation set

*Results to complete after provenance checks:* Present the four-class, two-foreground-class, and path-only comparisons on the documented lettuce evaluation set. Report per-class IoU and state which classes each average includes. Add a path-focused summary, identifying the checkpoint and split for each configuration. Keep validation exports separate from verified test results rather than combining their values.

### 3.8.4 Results on the selected operational subset

*Results to complete after image-list verification:* Present the path-only and four-class comparisons on the reported 42-image operational subset, alongside the manual reference. Identify the included folders and the checkpoints reused from the full-set evaluation. Interpret changes as differences between evaluated image groups, not improvements caused by retraining. The subset is not an independent additional test.

### 3.8.5 Discussion and limitations of the offline comparison

Training crop coverage, supervision source, and output classes must be kept distinct when interpreting the results. The manual model and the SAM-supervised models differ in more than annotation source: their training images also differ in crop coverage. Their performance gap cannot therefore be attributed solely to annotation quality. Likewise, a path-only model is not intrinsically more accurate because its label set matches the downstream task.

Claims about generalization require checking image overlap and related acquisitions. Performance on the selected operational subset does not establish the same accuracy across the full dataset. These offline comparisons do not establish geometric-reference accuracy, obstacle-free passage, or closed-loop navigation performance.

## 3.9 Segmentation configuration selected for robotic integration

The two selection decisions are complementary. The initial manual lettuce comparison motivates the architecture, LR-ASPP. The later multi-crop comparison concerns supervision and target classes, leading to the binary path configuration referred to as **allPlantsSAM**. The latter decision does not replace the architecture comparison.

The binary configuration was selected because its target is directly aligned with path extraction and its observed behavior on the selected operational images supported that choice. The quantitative evidence and limitations are to be completed in Section 3.8. This is not a claim that the binary configuration outperforms the manually supervised model or that performance on the selected rows holds across the entire dataset.

*Deployment record to complete:* Identify the exact trained checkpoint and exported model used by the robot, input resolution, preprocessing, and path-label encoding. Relate its output to the geometric interface in Chapter 4. Report quantitative comparisons only after their evaluation provenance has been established.

If test results contributed to selecting the deployed configuration, disclose that role: those images then served model selection rather than an entirely independent final assessment. Later geometric and field experiments must separately assess reference quality and the behavior of the integrated robot; segmentation IoU alone does not demonstrate successful row following.

## 3.10 ONNX export and inference configuration

### 3.10.1 Model export

### 3.10.2 Inference inputs and outputs

## Capitolo 4

# Geometric Extraction of the Local Navigation Reference

This chapter describes the two geometric extractors implemented in `pf_straight_prior3_single.py` and `pf_straight_prior4_single.py`. They are referred to as Method 3 and Method 4, respectively. Both convert a segmented corridor mask into a local navigation reference by extracting intervals in horizontal strips, associating their boundaries from bottom to top, and fitting the left and right edges separately. The reference is the mean of the two fitted edge lines, rather than a line fitted directly to interval midpoints.

The methods share mask preparation, interval extraction, and residual-based line reweighting. They differ in the support used to select the initial interval, the association cost, the handling of missing observations, and the checks applied before accepting the output. This chapter explains the active implementations and their default parameters; comments that describe inactive or intended behavior are not treated as implemented operations. Each image is processed independently. Temporal filtering, conversion to velocity commands, and closed-loop evaluation belong to the robotic integration stage.

### 4.1 Input representation and edge-line models

Both functions expect a two-dimensional label mask  $M(y, x)$  of height  $H$  and width  $W$ . Image coordinates increase to the right in  $x$  and downward in  $y$ . The argument `path_class` identifies the class from which corridor intervals are extracted; its default is 1 in both files. A binary input can therefore use 1 for the target region and 0 elsewhere. For a multiclass segmentation mask, this argument must match the actual label encoding, or the mask must first be converted appropriately.

The extractors do not infer the class mapping or use a separate plant class. A target-region label is not, by itself, a guarantee of traversable space.

The left and right boundaries are represented by

$$x_L(y) = m_L y + q_L, \quad x_R(y) = m_R y + q_R. \quad (4.1)$$

Treating  $x$  as a function of  $y$  permits vertical image lines without an infinite slope. The center reference is

$$x_C(y) = \frac{x_L(y) + x_R(y)}{2} = m_C y + q_C, \quad m_C = \frac{m_L + m_R}{2}, \quad q_C = \frac{q_L + q_R}{2}. \quad (4.2)$$

Thus, both methods assume locally straight image-space boundaries. The models do not reconstruct terrain or estimate a metric trajectory. Their residuals and width estimates are in pixels, not lateral distances in meters. The two edges need not be parallel: their separation can change linearly with image height.

At each image height, the reference lies halfway between the two predicted boundaries. This construction is not generally equivalent to fitting the observed interval midpoints: the two edge fits can exclude different border-touching samples and assign different residual weights. Keeping the boundaries separate allows each visible side to contribute its own observations, although both sides must have enough samples.

The shared processing stages are:

1. select a lower region of interest and clean the target-class mask;
2. extract and merge projected intervals in horizontal strips;
3. select a seed using the method-specific support and priority rules;
4. associate interval boundaries upward through the image;
5. fit the non-clipped left and right boundaries independently;
6. calculate quality indicators and rasterize the center reference, with method-specific validation and output rules.

## 4.2 Region of interest and mask preparation

The ROI begins at  $y_0 = \lfloor \rho H \rfloor$  and extends to the bottom of the image. The default starting ratio is  $\rho = 0.20$  for Method 3 and  $\rho = 0.30$  for Method 4. The binary target mask is

$$B(y, x) = \mathbf{1}\{M(y_0 + y, x) = \text{path\_class}\}, \quad 0 \leq y < H_R = H - y_0. \quad (4.3)$$

Both implementations apply opening with a  $5 \times 5$  elliptical kernel, then closing with a  $9 \times 9$  rectangular kernel. Connected components with fewer than 500 pixels are removed; all sufficiently large components are retained, not only the largest one.

Let  $\tilde{B}$  denote the cleaned binary ROI mask. It supplies both interval extraction and local support in the new methods. Unlike the previous midpoint-based implementation, these support calculations do not use the original multiclass mask to compute separate soil and plant ratios. Method 3 additionally consults the original input labels when assessing the rasterized centerline. Method 4 does not perform that centerline-label assessment.

Morphology and interval merging can bridge interruptions or change region connectivity. Consequently, support in  $\tilde{B}$  measures agreement with the processed mask, not independently verified scene geometry. Kernel sizes, component areas, and distance thresholds are in input-mask pixels and depend on image resolution and camera configuration.

## 4.3 Horizontal strips and candidate intervals

### 4.3.1 Strip construction and column projection

Both methods use  $N = 24$  strips by default, indexed from bottom to top. Their height is  $h = \max(1, \lfloor H_R/N \rfloor)$ . For strip index  $s$ , the ROI-relative bounds are

$$y_s^- = \max(0, H_R - (s + 1)h), \quad y_s^+ = \min(H_R, H_R - sh), \quad (4.4)$$

with the lower image row bound  $y_s^+$  excluded. Empty strips are skipped. The representative full-image coordinate is  $y_s = y_0 + (y_s^- + y_s^+)/2$ , following the code's bound-based convention. Integer division may leave a remainder above the constructed strips.

For example, with  $H_R = 157$  and  $N = 24$ , the strips cover the lowest  $24 \times 6 = 144$  ROI rows. The upper 13 ROI rows do not supply candidates; the ROI and the region actually sampled by the strips are therefore not identical.

Within each strip, the cleaned mask is summed vertically. A column is active whenever its sum exceeds zero, so even one target pixel activates that column. Contiguous active columns form raw intervals. Neighboring intervals are merged when the number of inactive columns between them is at most `max_interval_gap`, whose default is 30 pixels. For a merged interval  $[\ell_i, r_i]$ , the width is

$$d_i = r_i - \ell_i + 1. \quad (4.5)$$

Intervals narrower than 8 pixels are discarded. There is no fixed maximum interval width in either new method. Instead, an interval touching both image margins

is rejected if the mean of the entire cleaned binary strip is below 0.70. With `border_margin=2`, the flags are

$$t_{L,i} = \mathbf{1}\{\ell_i \leq 2\}, \quad t_{R,i} = \mathbf{1}\{r_i \geq W - 1 - 2\}. \quad (4.6)$$

These flags indicate possible truncation by the image boundary, rather than verified physical corridor edges.

Column projection merges evidence across different vertical locations; a merged interval can also contain horizontal gaps. Neither its width nor its endpoints prove that its rectangular interior is entirely target class. This matters when interpreting intervals around occlusions or fragmented segmentation.

### 4.3.2 Method-specific local support

Method 3 computes central support  $u_i$  in a band around the rounded interval midpoint. With default half-width 5, the band contains up to 11 columns and spans the strip height. Its value is the mean of  $\tilde{B}$  inside the band, clipped at the image boundaries. Candidates must satisfy  $u_i \geq 0.27$  for both seed selection and subsequent association.

This central band checks whether the projected interval contains target pixels near its midpoint, which is useful when merging has joined two regions across a gap. It does not check the whole interval or establish that both endpoints are reliable.

Method 4 instead computes  $u_{L,i}$  and  $u_{R,i}$  in separate bands centered on  $\ell_i$  and  $r_i$ . The effective half-width is  $\max(2, \text{local\_band\_half\_width})$ , equal to 5 by default. These means quantify target-pixel occupancy around each boundary; they are not boundary-detection probabilities. Because a band straddles a boundary, it need not be entirely target class even for a clean interval.

In Method 4, edge support ranks seed candidates and contributes to the final quality score. It is not an acceptance threshold and does not weight the line fit. In particular, `min_edge_support=0.20` is present in the signature but unused in the active implementation.

## 4.4 Initialization and bottom-up association

### 4.4.1 Method 3: lowest supported seed

Method 3 searches the first six constructed strips from the bottom, or all available strips if fewer exist. It selects the lowest strip with at least one candidate satisfying  $u_i \geq 0.27$ . Within that strip, it minimizes distance between the interval midpoint and  $(W - 1)/2$ ; ties favor the widest interval. There is no consecutive-run eligibility test.

Let  $\ell_{\text{prev}}, r_{\text{prev}}, d_{\text{prev}}$  denote the last accepted interval. In each strip above the seed, define

$$\delta_i = \max(|\ell_i - \ell_{\text{prev}}|, |r_i - r_{\text{prev}}|), \quad D_3 = \max(42, 0.5d_{\text{prev}}). \quad (4.7)$$

Among candidates with adequate central support and  $\delta_i \leq D_3$ , the method minimizes

$$J_{3,i} = \delta_i + 0.1|d_i - d_{\text{prev}}|. \quad (4.8)$$

A strip without an admissible candidate is skipped, leaving the previous interval unchanged. There is no maximum number of consecutive missed strips, and the jump threshold is not increased to account for a longer vertical gap. Strips below the selected seed are not revisited.

#### 4.4.2 Method 4: edge-support seed and gap-limited association

Method 4 pools all candidates in the first six constructed strips. It maximizes  $\min(u_{L,i}, u_{R,i})$ , then interval width. Unlike Method 3, it does not prioritize proximity to the image center or explicitly choose the lowest eligible strip. Exact ties retain the first candidate in the bottom-up, left-to-right traversal. No minimum-support threshold is applied, so even a weakly supported candidate can initialize tracking.

The boundary displacement  $\delta_i$  is defined as above, but the jump limit and association cost are

$$D_4 = \max(28, 0.5d_{\text{prev}}), \quad J_{4,i} = \delta_i + 0.4|d_i - d_{\text{prev}}|. \quad (4.9)$$

Only the displacement limit determines admissibility; edge support does not enter this cost or an additional tracking gate. Compared with Method 3, the default lower bound on allowable displacement is smaller and width change has a larger cost coefficient. For wide intervals, the width-dependent term can dominate the jump limit in either method.

A counter records consecutive strips without a compatible candidate and resets after each successful association. With `max_gap_strips=3`, tracking stops on the fourth consecutive miss. Stopping does not directly return `lost`: the accumulated points proceed to fitting if at least three are available. The maximum miss count, including the miss that triggers stopping, enters the quality score. The previous interval remains unchanged during gaps.

The stopping rule and gap penalty have different effects: three misses are tolerated for continued association, but already reduce the gap score to zero. A fourth miss stops the search without automatically rejecting a reference supported by the observations collected earlier.

Both methods perform greedy spatial association within one frame, not temporal tracking. Each accepted interval contributes two boundary observations, and at least three accepted intervals are required before the edge-fitting stage.

## 4.5 Separate robust fitting of the corridor edges

### 4.5.1 Excluding image-boundary observations

The left fit uses only accepted intervals not flagged as touching the left image margin. The right fit independently excludes intervals flagged as touching the right margin. At least two observations per edge must remain. The two fits can therefore use different strips and span different vertical ranges. A clipped boundary is not replaced by a one-sided centerline estimate: if either edge lacks sufficient samples, both implementations return an empty output with `lost` status.

The retained samples are interval endpoints from the cleaned column projection. They are not points obtained by a dedicated edge detector. Excluding border-touching samples reduces the influence of image truncation, but does not establish the correctness of the remaining boundaries.

### 4.5.2 Residual-based reweighting

Both methods start each edge fit with uniform weights. There is no semantic, interval-width, or vertical-position base weighting. For an edge sample  $(x_i, y_i)$ , four reweighting iterations are performed by default, with scale  $\sigma = 18$  pixels:

$$e_i^{(t)} = x_i - m^{(t)}y_i - q^{(t)}, \quad w_i^{(t+1)} = \frac{1}{1 + (e_i^{(t)}/\sigma)^2}. \quad (4.10)$$

Each iteration calls `numpy.polyfit` with degree one and updated weights. Its weights multiply unsquared residuals [9]; the fitted objective at that iteration is

$$(m, q) = \arg \min_{m, q} \sum_i [w_i(x_i - my_i - q)]^2. \quad (4.11)$$

Thus, the effective coefficient of each squared residual is  $w_i^2$ , not  $w_i$ . The implementation should not be interpreted as a different standard robust-loss estimator solely from the form of its weight rule. Weights are recomputed from the current model rather than accumulated across iterations; no convergence-based stopping criterion is used. Method 3 additionally checks for negligible total weights in its helper.

The final residual for each retained sample is its absolute horizontal deviation from the corresponding fitted edge. Define  $\bar{e}$  as the mean over the concatenated left and right residual arrays, and

$$F = \text{clip}(1 - \bar{e}/\sigma, 0, 1). \quad (4.12)$$

Edges with more retained samples contribute more terms to this mean. Reweighting reduces the influence of inconsistent points but cannot correct an erroneous association chain that supports a coherent wrong line. It does not impose parallel edges or estimate genuine path curvature.

## 4.6 Quality indicators, validation, and image-space output

### 4.6.1 Method 3: centerline support after rasterization

Method 3 constructs the mean edge line and rasterizes it from the top of the highest accepted strip to the exclusive lower bound of the seed strip. It does not extend the line below an elevated seed. Within this range, it fills rows across skipped strips as well. Predicted horizontal positions are clipped to  $[0, W - 1]$  and converted to integers; dilation with a  $3 \times 3$  elliptical kernel thickens the line. Dilation may extend the nonzero output slightly beyond the pre-dilation row range.

Before dilation, the original input labels are checked in a band of half-width 3 around each rasterized point. Let  $P$  be the fraction of pixels equal to `path_class` across these clipped bands, and  $T$  the number of accepted intervals divided by the number of constructed strips. The score is

$$C_3 = 0.50T + 0.30P + 0.20F. \quad (4.13)$$

The status is `lost` if  $C_3 < 0.38$  or  $T < 0.25$ , `degraded` if neither condition holds but  $C_3 < 0.60$ , and `ok` otherwise. These checks follow rasterization: a result marked `lost` can still contain a nonempty centerline matrix. No explicit edge-order or positive-width test is applied.

The diagnostic dictionary includes both edge models, the centerline model, row bounds, residual and support indicators, clipped-point counts, and the cleaned full-image ROI mask. Early selection or fitting failures instead return a zero matrix and a reason string, with additional details in some branches.

### 4.6.2 Method 4: geometric checks and a composite score

Method 4 validates the fitted boundaries before constructing the output. At 16 evenly spaced heights covering the combined range of retained edge samples, it evaluates

$$D(y) = x_R(y) - x_L(y). \quad (4.14)$$

All sampled widths must be strictly positive. The median of these sampled widths is returned as `width`. This check concerns the sampled observation range, not any extrapolation below it.

A separate indicator uses all accepted intervals, including those with border-touching endpoints. For  $K$  intervals in association order, let  $z_j = r_j - \ell_j$ ; unlike extraction width  $d_j$ , this quantity omits the inclusive-endpoint  $+1$ . The width-irregularity indicator is

$$E_D = \frac{Q_{0.90} \left( \{|z_{j+2} - 2z_{j+1} + z_j|\}_{j=1}^{K-2} \right)}{\max(\text{median}_j |z_j|, 1)}, \quad A = \text{clip}(1 - E_D/0.30, 0, 1), \quad (4.15)$$

where  $Q_{0.90}$  is the 90th percentile. These are second differences in sample order, not derivatives normalized by vertical distance. If strips are skipped, sample spacing is unequal; even a width varying linearly with image height need not yield zero second differences. The indicator therefore measures an implementation-specific irregularity, not a perspective-invariant physical curvature.

Let  $S$  be the mean of the left support values used by the left fit and the right support values used by the right fit, pooled together. With maximum consecutive miss count  $g_{\max}$  and allowed gap  $g_a = 3$ ,

$$G = \text{clip}(1 - g_{\max} / \max(g_a, 1), 0, 1), \quad C_4 = 0.25T + 0.20S + 0.25F + 0.20A + 0.10G. \quad (4.16)$$

The result is **lost** if the positive-width check fails,  $\bar{e} > 2\sigma$ , or  $C_4 < 0.38$ . Otherwise, it is **degraded** for  $C_4 < 0.60$  and **ok** for  $C_4 \geq 0.60$ . There is no separate  $T < 0.25$  rejection rule in Method 4.

A result marked **lost** returns a zero output matrix before rasterization, with no centerline model in the post-fit rejection branch. For an accepted result, the mean edge line is rasterized from the top of the highest accepted strip to the bottom of the image, then dilated as in Method 3. This can extrapolate below the seed. It does not check target labels along the rendered centerline, and the edge-order check does not cover the entire rendered range when that range exceeds the sampled observation range.

Diagnostics report the edge and centerline models, mean residual, support, tracking fraction, median fitted width, width irregularity, maximum gap, row bounds, and cleaned ROI mask. Earlier failures return **lost** with reason strings such as `no_seed_candidate`, `tracking_failed`, or `insufficient_edge_points`.

### 4.6.3 Interpretation of status and confidence

The two confidence values are heuristic scores with different components; they are not calibrated probabilities and should not be compared as if an identical numerical value represented identical reference quality. Method 3 assesses original-mask support along the centerline, whereas Method 4 combines processed-mask edge support with geometric and gap indicators. Neither verifies obstacle-free passage.

A downstream controller must inspect the returned status. In particular, a nonempty Method 3 output does not necessarily denote accepted guidance. Both methods can also clip an out-of-image prediction onto an image border. Neither function specifies a robot stopping or recovery policy. Malformed inputs, unsuitable parameter values, or numerical exceptions are not comprehensively handled by the explicit failure branches.

## 4.7 Default configurations and comparison

Table 4.1 distinguishes shared defaults from method-specific settings. These are the defaults of the supplied files, not a statement that a controller uses them or that they are experimentally optimal. Overrides and mask encodings must be recorded for each experiment.

**Tabella 4.1:** Default configurations of the two edge-based extractors. Distances and areas are expressed in input-mask pixels.

| Setting                            | Method 3                    | Method 4           |
|------------------------------------|-----------------------------|--------------------|
| Target-class identifier            | 1                           | 1                  |
| ROI starting ratio                 | 0.20                        | 0.30               |
| Number of strips                   | 24                          | 24                 |
| Opening / closing kernels          | $5 \times 5$ / $9 \times 9$ | Same               |
| Minimum component area             | 500                         | 500                |
| Minimum interval width             | 8                           | 8                  |
| Maximum merged gap                 | 30                          | 30                 |
| Image-border margin                | 2                           | 2                  |
| Full-strip occupancy gate          | 0.70                        | 0.70               |
| Local support-band half-width      | 5                           | 5                  |
| Seed search window                 | 6 strips                    | 6 strips           |
| Central support threshold          | 0.27                        | Not used           |
| Minimum edge support threshold     | Not used                    | Inactive parameter |
| Jump-limit lower bound             | 42                          | 28                 |
| Width-change cost coefficient      | 0.1                         | 0.4                |
| Consecutive misses allowed         | No fixed limit              | 3                  |
| Residual scale / iterations        | 18 / 4                      | 18 / 4             |
| Centerline assessment half-width   | 3                           | Not used           |
| Width-error score scale            | Not used                    | 0.30               |
| Lost / acceptable score thresholds | 0.38 / 0.60                 | 0.38 / 0.60        |
| Minimum tracking fraction gate     | 0.25                        | Not used           |
| Output dilation kernel             | $3 \times 3$                | $3 \times 3$       |

The principal difference is not the line model but the selection and validation surrounding it. Method 3 favors a supported interval near the image center in the lowest eligible strip, accepts arbitrarily long association gaps, and evaluates the rendered line against the original mask. Method 4 favors joint support of the two boundaries across the bottom search window, limits consecutive gaps, penalizes width changes more strongly, and rejects outputs failing its geometric or score checks. These choices can affect different failure cases, but they do not establish that one method is more accurate or faster without evaluation.

Both methods can merge separated regions, follow an incorrect but consistent interval chain, or infer a line through an unobserved gap. A wide or well-supported interval is not necessarily the intended row. The two-line representation retains boundary information separately, but requires sufficient non-clipped samples on both sides. Independent edge fits may be supported over different height ranges, and their mean can include extrapolation of one or both boundaries.

Evaluation should separately assess interval association, edge fitting, centerline accuracy, rejection behavior, and closed-loop motion. A controlled comparison should use the same input masks and first align shared settings such as ROI and strip count, so differences are not attributed solely to algorithmic rules when input regions also differ. Useful component comparisons include seed priority, width-change cost, gap stopping, and post-fit validation. The descriptions here specify implemented behavior, not results of those comparisons.

## 4.8 Effect of strip height and number

The strip count controls vertical sampling and the maximum number of interval observations. More strips provide thinner vertical regions and potentially more points per boundary fit. Taller strips aggregate more mask pixels but mix evidence from different image heights, and column projection can conceal interruptions. Increased strip height does not guarantee more reliable boundary estimates or a numerically better fit.

For a  $224 \times 224$  input, Method 3 has  $y_0 = 44$  and  $H_R = 180$ ; Method 4 has  $y_0 = 67$  and  $H_R = 157$ . Table 4.2 uses the exact integer-division rule for these illustrative dimensions, rather than assuming the same ROI height for both methods. At  $N = 24$ , the strip heights are 7 and 6 pixels, respectively. A fit can use fewer than  $N$  points because of seed location, missed strips, early stopping, or border-point exclusion; the two edges can have different sample counts.

Changing  $N$  also changes the vertical extent of a fixed six-strip seed window and, in Method 4, the extent of a three-strip tolerated gap. These settings should therefore be considered together. The width-irregularity indicator depends on the sampling sequence, while tracking fractions use the number of constructed strips as

**Tabella 4.2:** Strip heights for an illustrative  $224 \times 224$  mask and the default ROI ratios. The point count is an upper bound per edge, not a guaranteed number of fitted samples.

| Strips $N$ | Method 3 height | Method 4 height | Max. edge samples |
|------------|-----------------|-----------------|-------------------|
| 24         | 7 px            | 6 px            | 24                |
| 20         | 9 px            | 7 px            | 20                |
| 16         | 11 px           | 9 px            | 16                |
| 12         | 15 px           | 13 px           | 12                |
| 8          | 22 px           | 19 px           | 8                 |

their denominator. Selecting  $N$  requires evaluating boundary and centerline errors, accepted-point counts, rejection rates, and processing time under the intended image resolution and field conditions. No optimal strip count is inferred from the implementation alone.

## Capitolo 5

# Conclusions and Future Work

This chapter will synthesize the findings on dataset preparation, segmentation, geometric reference extraction, and robot operation. It is currently a working outline: the completion notes identify the material to add after experimental evaluation and do not report tests or results already obtained. Detailed protocols, numerical tables, and individual trial results belong in the experimental chapter; the conclusions should interpret that evidence without repeating it in full.

### 5.1 Thesis objectives and main contributions

*To complete:* Return to the objectives stated in the introduction and summarize the work actually completed. Distinguish implementation contributions from experimentally demonstrated benefits.

- Dataset preparation and the use of manual and SAM 3-generated annotations for agricultural segmentation.
- Selection of a segmentation architecture and of the trained configuration used for robotic integration.
- Implementation and comparison of the two edge-based geometric extractors described in Chapter 4.
- Integration of perception, reference extraction, and control, limited to the functionality and validation actually completed.

## 5.2 Conclusions on dataset preparation and automatic annotation

*To complete:* Explain what the experiments establish about replacing or reducing manual annotation. Keep direct mask-quality evaluation separate from the performance of networks trained on those masks.

- Summarize the effects of manual and automatic supervision on path segmentation, distinguishing the full evaluation set from the selected operational subset.
- Discuss the role of label sets and prompt targets, including visible soil versus a central navigation-path annotation.
- State whether annotation-time savings were measured, including inspection and correction; do not infer savings from model IoU.
- Identify the conditions in which automatic labels were useful and the cases requiring manual inspection or revision.

## 5.3 Selection of the segmentation model for the robot

### 5.3.1 Architecture selection: accuracy and inference time

*To complete:* Synthesize the initial FCN, DeepLabV3, and LR-ASPP comparison on manually annotated lettuce. Explain the practical choice of LR-ASPP using measured accuracy and inference time under documented, comparable conditions. Distinguish training duration from inference latency, and model-only inference from end-to-end robot processing.

### 5.3.2 Supervision and target-class selection

*To complete:* Explain the separate decision to use the binary `allPlantsSAM` configuration. Relate its path-focused output to the geometric extractor and summarize the evidence supporting its use in the selected operating conditions. Do not describe it as the most accurate configuration if the manually supervised model performs better. State whether evaluation results also contributed to this selection.

## 5.4 Conclusions on geometric reference extraction

*To complete:* Compare Method 3 and Method 4 using the evidence from the geometric experiments, rather than their design rules alone.

- Discuss interval association, independent boundary fitting, and centerline accuracy on the same evaluation masks.
- Summarize the effects of seed selection, association gaps, border-point exclusion, and post-fit validation.
- Report the implications of strip count, ROI, and parameter choices, distinguishing controlled comparisons from default settings.
- Discuss rejection behavior and incorrect but geometrically consistent references; a fitted line is not verified free space.

## 5.5 Conclusions from robot field validation

*To complete after the field campaign:* Summarize what closed-loop trials establish beyond offline image segmentation and line extraction. If field validation is not completed, explicitly state that limitation instead of writing this section as if the trials had taken place.

### 5.5.1 Test conditions and scope of the evidence

*To complete:* Briefly identify the robot, crop and row conditions, camera configuration, deployed checkpoint, geometric method, controller, and number and length of trials. Refer to the experimental chapter for full configuration and trial records. Distinguish any planned scenarios from those actually tested, including illumination changes, discontinuous rows, partial occlusions, and different initial offsets.

### 5.5.2 Row following and task completion

*To complete:* Interpret path-following accuracy and the fraction of trials completed under a predefined success criterion. Summarize travelled distance, failures, and operator interventions as well as successful runs. Define how lateral and heading errors were measured; image-coordinate deviations must not be presented as metric cross-track errors without an independent measurement or justified conversion.

### 5.5.3 Reference loss, rejection, and robot response

*To complete:* Discuss the observed behavior when the extractor returns **degraded** or **lost**, when the reference becomes incorrect, or when observations are interrupted. Relate these states to the controller’s implemented slowing, stopping, or recovery behavior and to any manual intervention. Describe the supervision and protective measures actually used in the tests. Do not infer collision avoidance from a confidence score or from the presence of a centerline.

### 5.5.4 Processing latency and closed-loop operation

*To complete:* Relate measured segmentation and geometric-processing latency to the actual control update rate and robot speed. Include end-to-end delay, variability, and dropped or stale observations where recorded. A short network inference time alone does not establish that the integrated system meets its timing requirements.

## 5.6 Limitations and validity of the findings

*To complete:* Identify the limits supported by the final experiment records. Resolve outstanding checks before treating them as verified facts.

- Evaluation-set provenance, including the distinction between validation exports, full-test results, and the operational subset.
- Potential overlap, duplicate images, or related acquisitions between training and evaluation datasets.
- Any use of test results for model selection and the resulting limits on independent final evaluation.
- Sensitivity to label definitions, segmentation errors, image resolution, camera pose, and extractor parameters.
- Locally straight image-space boundaries, extrapolation, and the absence of verified obstacle detection or terrain reconstruction.
- Field-trial scope, measurement uncertainty, and the distinction between offline accuracy and closed-loop navigation performance.

## 5.7 Future work

### 5.7.1 Dataset expansion and annotation verification

*To complete:* Prioritize additional crop and lighting conditions, manual review of difficult automatic masks, annotation-time measurement, and acquisition-separated evaluation sets. Explain which observed limitation motivates each proposed extension.

### 5.7.2 Geometric extraction and temporal integration

*To complete:* Discuss extensions justified by the measured failure cases, such as parameter scaling, adaptive strips, temporal association, curved references, or improved handling of partially visible boundaries. Clearly distinguish these proposals from implemented functionality.

### 5.7.3 Further robot trials and operational coverage

*To complete:* Identify the field tests still needed to extend the evidence: repeated trials across rows and acquisition sessions, longer runs, additional initial offsets, and conditions not represented in the completed campaign. Where relevant, propose evaluating reference-loss handling and controller behavior under supervised, controlled conditions. Do not claim operational coverage beyond the environments actually tested.

## 5.8 Final remarks

*To complete last:* Provide a concise answer to the thesis objectives using only findings established by the final evidence. State the level of validation reached and the main remaining limitation. Avoid introducing new experiments, unsupported performance claims, or results not discussed in the preceding chapters.

# Bibliografia

- [1] Yuhao Bai, Baohua Zhang, Naimin Xu, Jun Zhou, Jiayou Shi e Zhihua Diao. «Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review». In: *Computers and Electronics in Agriculture* 205, 107584 (feb. 2023). DOI: 10.1016/j.compag.2022.107584 (cit. a p. 3).
- [2] Jiqing Chen, Hu Qiang, Jiahua Wu, Guanwen Xu e Zhikui Wang. «Navigation Path Extraction for Greenhouse Cucumber-Picking Robots Using the Prediction-Point Hough Transform». In: *Computers and Electronics in Agriculture* 180, 105911 (gen. 2021). DOI: 10.1016/j.compag.2020.105911 (cit. alle pp. 3, 5, 7, 8).
- [3] Rajitha de Silva, Grzegorz Cielniak, Gang Wang e Junfeng Gao. «Deep Learning-Based Crop Row Detection for Infield Navigation of Agri-Robots». In: *Journal of Field Robotics* 41.7 (2024), pp. 2299–2321. DOI: 10.1002/rob.22238 (cit. alle pp. 3, 4, 6–8).
- [4] Enrico Mendez, Javier Piña Camacho, Jesús Arturo Escobedo Cabello e Alfonso Gómez-Espinosa. «Autonomous Navigation and Crop Row Detection in Vineyards Using Machine Vision with 2D Camera». In: *Automation* 4.4 (set. 2023), pp. 309–326. DOI: 10.3390/automation4040018 (cit. alle pp. 3–8).
- [5] Alessandro Navone, Mauro Martini, Marco Ambrosio, Andrea Ostuni, Simone Angarano e Marcello Chiaberge. «GPS-Free Autonomous Navigation in Cluttered Tree Rows with Deep Semantic Segmentation». In: *Robotics and Autonomous Systems* 183, 104854 (gen. 2025). DOI: 10.1016/j.robot.2024.104854 (cit. alle pp. 3–8).
- [6] Johannes Kneip, Patrick Fleischmann e Karsten Berns. «Crop Edge Detection Based on Stereo Vision». In: *Robotics and Autonomous Systems* 123, 103323 (gen. 2020). DOI: 10.1016/j.robot.2019.103323 (cit. alle pp. 3–5, 7, 8).
- [7] Nicolas Carion et al. «SAM 3: Segment Anything with Concepts». In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2511.16719 [cs.CV]. URL: [https://proceedings.iclr.cc/paper\\_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html) (cit. a p. 5).

- [8] Xia Li, Junhao Su, Zhenchao Yue e Fangtao Duan. «Adaptive Multi-ROI Agricultural Robot Navigation Line Extraction Based on Image Semantic Segmentation». In: *Sensors* 22:20, 7707 (ott. 2022). DOI: 10.3390/s22207707 (cit. alle pp. 5, 7).
- [9] NumPy Developers. *numpy.polyfit: NumPy v2.4 Manual*. URL: <https://numpy.org/doc/2.4/reference/generated/numpy.polyfit.html> (visitato il giorno 13/09/2026) (cit. a p. 28).